

Week 3

Aim

The aim of this workshop was to learn more about hashing and authentication works whether it was through two factor authentication or password validation. The workshop also builds towards understanding how hashing works specifically with salt and pepper and how to secure these sensitive credentials safely. Additionally, we were tasked with completing coding exercises which allowed us to get hands-on experience on how these applications are executed.

Glossary

Brute force attack – A type of attack used that allows for someone to use a range of combinations until it matches correct credentials and they are permitted access.

MD5 – A hash function that produces 128-bit value

SHA-256 – Another hash function that uses 256-bit hash

Salt – A random value that is added to a password before hashing

Pepper – A secret value that is added to passwords because of hashing, but it is not stored with the password.

Hashing – A one-way mathematical function that converts data into a fixed size output.

Password strength – A measure of how hard a password is to guess

Discussion Points

Section 1

- Run the code and test various passwords. Look at the feedback for "Pass123" vs. "MyP@ssw0rd". Why is the second one significantly better?

The second one is better because Pass123 is considered weak, this is because it is short, common, has a low character pool and therefore it makes it easier to guess. Whilst on the other hand, 'MyP@ssw0rd' is stronger because it has much more entropy since it uses a wider range of characters.

- What are the limitations of this checker? (e.g., It doesn't detect dictionary words like "CorrectHorseBatteryStaple" or keyboard patterns like "asdfghjkl").

The password checker does not detect common keyboard patterns; it also does not detect repeated characters and only checks for exact matches in a common password list. It also does not detect dictionary words.

Section 2

- how the MD5 and SHA-256 hashes are just short strings.

They are both traditional hash functions. However, no matter the length of the input, MD5 produces a 128-bit hash whilst sha256 provides 256-bit hash.

- The bcrypt hash looks different: \$2b\$12\$.... It contains the algorithm version, the work factor (12), the salt, and the hash all in one string

A bcrypt hash bundles multiple pieces of information into one string, so it can be verified later. Bcrypt hashes are generally longer because it stores metadata needed to verify the hash, making it safer and more flexible than MD5 or SHA-256.

- Question: Why is a fast algorithm bad for storing passwords? (It makes brute-force attacks much faster for an attacker who steals the hash database).

A fast-hashing algorithm is bad for password storage since it allows for attackers to perform brute force and dictionary attacks extremely quickly. Algorithms like MD5 or SHA-256 are designed to be fast but are not effective for password security because they can allow for a threat actor to try many guesses per second.

Section 3: Discussion points

If an attacker steals your database, what can they do if:

- You only used SHA-256 with no salt? (Look up all the hashes in a rainbow table).

It means that every user who has the same password will also have the same hash, making it easier to look up the hash in a rainbow table and guess weak passwords.

- You used SHA-256 with a unique salt for each user? (They can't use a rainbow table, but they still have to attack each password individually. This is much slower!).

It will result in a more secure protection of passwords but still vulnerable since SHA-256 is fast however it slows an attacker down dramatically.

- You used SHA-256 with salt AND a pepper? (They can't begin cracking passwords until they also find and steal the secret pepper).

A pepper is a secret value stored separately from the database with both salt and pepper and it results in a much more secure system because the attacker then needs both the pepper and the database before cracking.

Section 4: Discussion Points

- Question: Why is TOTP considered more secure than 2FA codes sent via SMS? (SMS messages can be intercepted, and phone numbers can be stolen via "SIM swapping" attacks)

TOTP is more secure because attacks can intercept SMS over insecure networks. Alongside this, attacks can use malware and spyware on the phone and exploit weaknesses in mobile phone carriers such as through SIM swapping.

Section 5: Discussion Points

- Observe how quickly the MD5 and SHA-256 hashes for "password" are cracked (likely in milliseconds).
- The demo doesn't even try to crack the bcrypt hash. Why?

Bcrypt is deliberately slow, and that slowness is its main security feature by making large scale cracking extremely expensive.

Issues/Errors

Password strength using pool size and entropy. One of the key issues I ran into during the project was understanding pool size and entropy and how that affected the passwords' strength. After some deeper research though I understand how it can be used to measure how unpredictable a password is. Understanding the formula for entropy $\text{length} \times \log^2(\text{pool size})$ and how this translates into security. Entropy is a measurement of unpredictability of passwords, and the higher an entropy is the harder the password is to guess; it is the length of the passwords multiplied by the number of different characters that could be used.

Key Takeaways

Strong password design matters – Through having a strong password, you are able to make it harder for attacks to guess your password especially if they are using social engineering techniques and looking into you and attempts to guess your password.

Therefore, having a strong password design will always be an extra layer of protection for users.

Never store plaintext password – Hashing is irreversible and having bcrypt is ideal for protecting passwords. Through storing passwords in plain texts, you leave yourself extremely vulnerable to an attack.

Salt and pepper dramatically improve security – Having salt helps prevent attacks such as rainbow table attacks and through pepper it adds an extra layer even if the database is stolen.

2FA strengthens authentication – It is more secure than SMS since SMS can be intercepted or stolen and therefore having it being hosted on an app prevents events such as sim swapping from occurring.